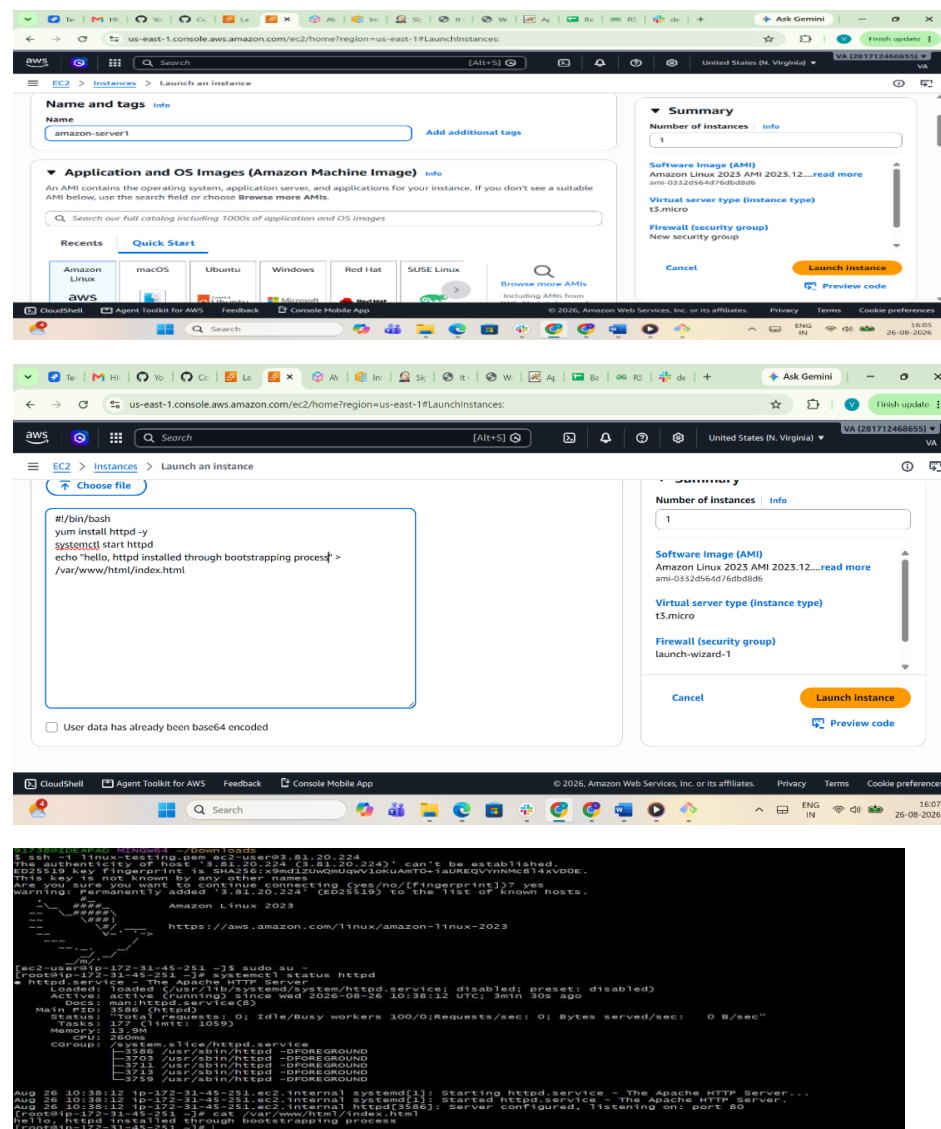


Task on EC2

1. Launch one EC2 using Amazon Linux 2 image and add a script in user data to install Apache.

Implementation:

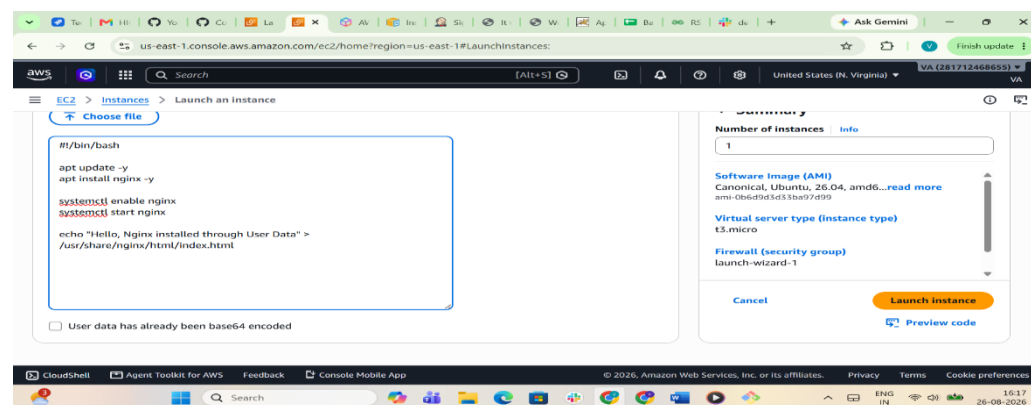
1. Launched an EC2 instance using the Amazon Linux 2 AMI.
2. Selected the required instance type, key pair, subnet, and security group.
3. Added a user data script to install and start the Apache web server during instance launch.
4. Connected to the EC2 instance using SSH.
5. Verified that Apache was installed and running.
6. Tested the Apache default page using the EC2 public IP.



2. Launch one EC2 using Ubuntu image and add a script in user data to install Nginx.

Implementation:

1. Launched an EC2 instance using the Ubuntu AMI.
2. Configured the instance type, key pair, subnet, and security group.
3. Added a user data script to install and start the Nginx web server automatically.
4. Connected to the Ubuntu EC2 instance using SSH.
5. Verified that Nginx was installed and running.
6. Accessed the Nginx default page using the EC2 public IP.



```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-36-45:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-08-26 10:48:11 UTC; 4min 26s ago
     Invocation: 723ec090e2f149a8814ff4346c778def
       Docs: man:nginx(8)
    Main PID: 1516 (nginx)
      Tasks: 3 (limit: 627)
     Memory: 3M (peak: 7.2M)
        CPU: 43ms
      CGroup: /system.slice/nginx.service
              └─1516 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
                └─1519 "nginx: worker process"
                  └─1520 "nginx: worker process"

Aug 26 10:48:11 ip-172-31-36-45 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Aug 26 10:48:11 ip-172-31-36-45 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-36-45:~$ cat /usr/share/nginx/html/index.html
Hello, Nginx installed through User Data
ubuntu@ip-172-31-36-45:~$
```

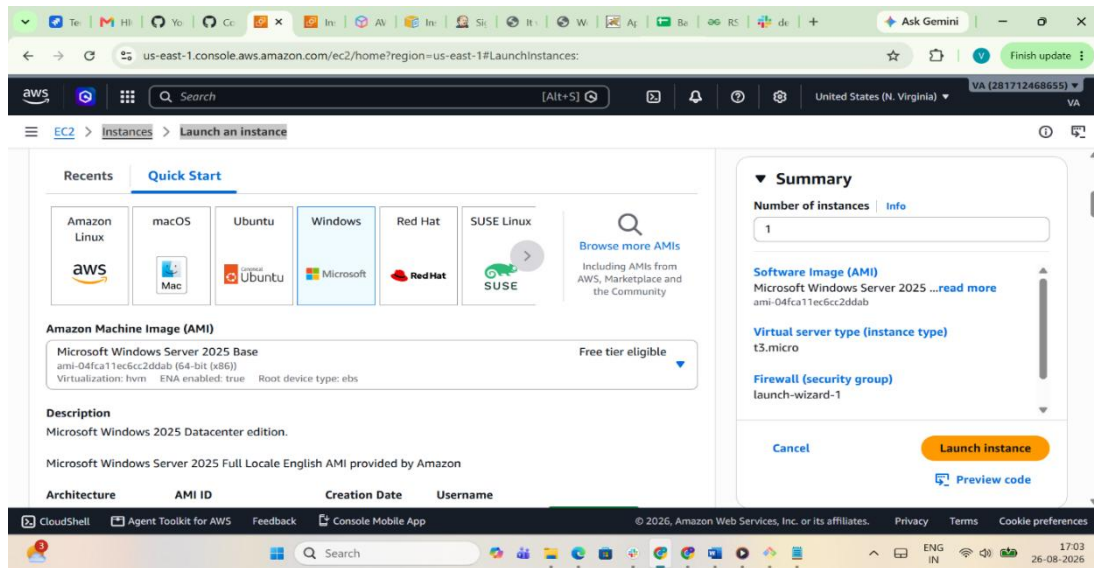
3. Launch one Windows server and install Tomcat on Windows.

Implementation:

1. Launched an EC2 instance using a Windows Server AMI.
2. Configured the instance type, key pair, subnet, and security group.
3. Connected to the Windows EC2 instance using Remote Desktop Protocol (RDP).
4. Installed Java on the Windows server.
5. Downloaded and extracted Apache Tomcat.

6. Configured Tomcat and started the Tomcat server.

7. Verified the Tomcat server using port 8080.



```
PS C:\Windows\system32> cd C:\tomcat
PS C:\tomcat> Get-ChildItem

Directory: C:\tomcat

Mode                LastWriteTime         Length Name
----                -
-a-----         8/26/2026  12:06 PM          13719883 apache-tomcat-9.0.121.zip

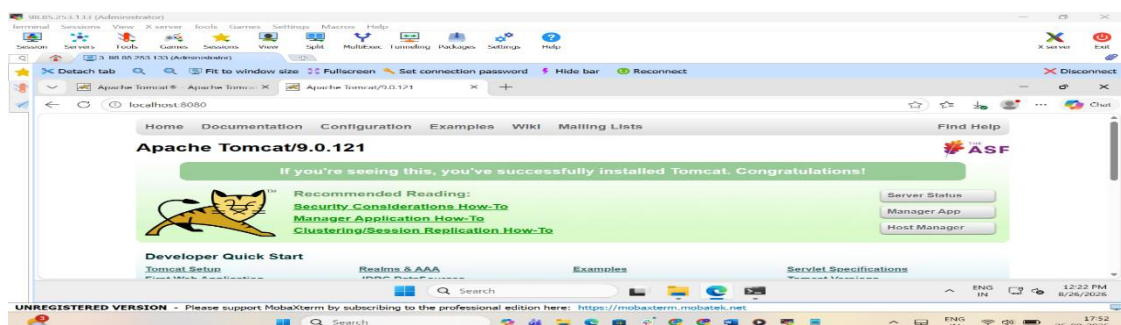
PS C:\tomcat> Expand-Archive -Path .\apache-tomcat-9.0.121.zip -DestinationPath .
PS C:\tomcat> Get-ChildItem

Directory: C:\tomcat
```

```
PS C:\tomcat> cd C:\tomcat\apache-tomcat-9.0.121\bin
PS C:\tomcat\apache-tomcat-9.0.121\bin> Get-ChildItem

Directory: C:\tomcat\apache-tomcat-9.0.121\bin

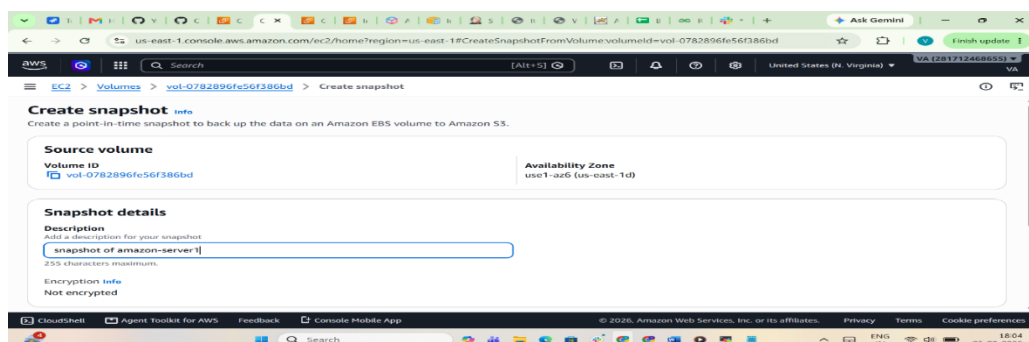
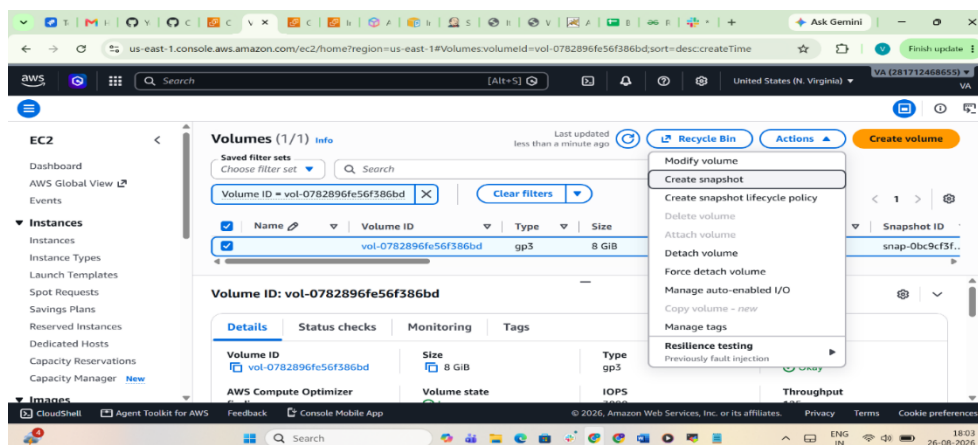
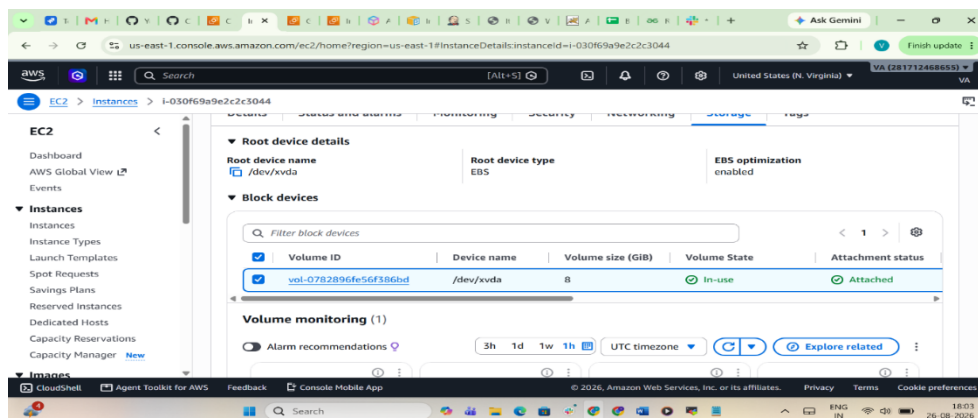
Mode                LastWriteTime         Length Name
----                -
-a-----         8/12/2026   7:38 PM          36230 bootstrap.jar
-a-----         8/12/2026   7:38 PM           1703 catalina-tasks.xml
-a-----         8/12/2026   7:38 PM          17614 catalina.bat
-a-----         8/12/2026   7:38 PM          26029 catalina.sh
-a-----         8/12/2026   7:38 PM           2123 ciphers.bat
-a-----         8/12/2026   7:38 PM           1997 ciphers.sh
-a-----         8/12/2026   7:38 PM         222622 commons-daemon-native.tar.gz
-a-----         8/12/2026   7:38 PM          27078 commons-daemon.jar
-a-----         8/12/2026   7:38 PM           2040 configtest.bat
-a-----         8/12/2026   7:38 PM           1922 configtest.sh
-a-----         8/12/2026   7:38 PM           9100 daemon.sh
-a-----         8/12/2026   7:38 PM           2091 digest.bat
```



4. Take a snapshot of the instance created in Task 1.

Implementation:

1. Identified the EC2 instance created in Task 1.
2. Identified the EBS volume attached to the instance.
3. Created an EBS snapshot of the attached volume.
4. Added a suitable snapshot description and tags.
5. Waited for the snapshot status to become completed.
6. Verified the snapshot in the EBS Snapshots section.




```

ubuntu@ip-172-31-36-45:~$ sudo find / -name authorized_keys
/home/ubuntu/.ssh/authorized_keys
/root/.ssh/authorized_keys
ubuntu@ip-172-31-36-45:~$ nano ^C
ubuntu@ip-172-31-36-45:~$ nano /home/ubuntu/.ssh/authorized_keys
ubuntu@ip-172-31-36-45:~$ exit
logout
Connection to 34.207.106.106 closed.

91738@IDEAPAD MINGW64 ~/Downloads
$ ssh ubuntu@34.207.106.106
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Aug 26 11:28:39 UTC 2026

System load:  0.02           Temperature:   -273.1 C
Usage of /:   34.4% of 6.61GB Processes:      120
Memory usage: 32%           Users logged in: 0
Swap usage:   0%            IPv4 address for ens5: 172.31.36.45

Expanded Security Maintenance for Applications is not enabled.

120 updates can be applied immediately.
90 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

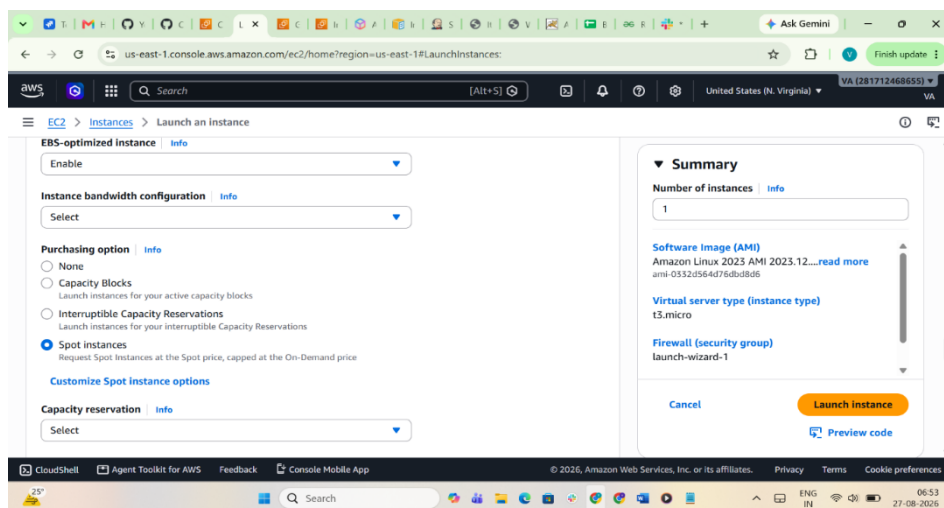
Last login: Wed Aug 26 11:24:46 2026 from 103.203.172.230
ubuntu@ip-172-31-36-45:~$

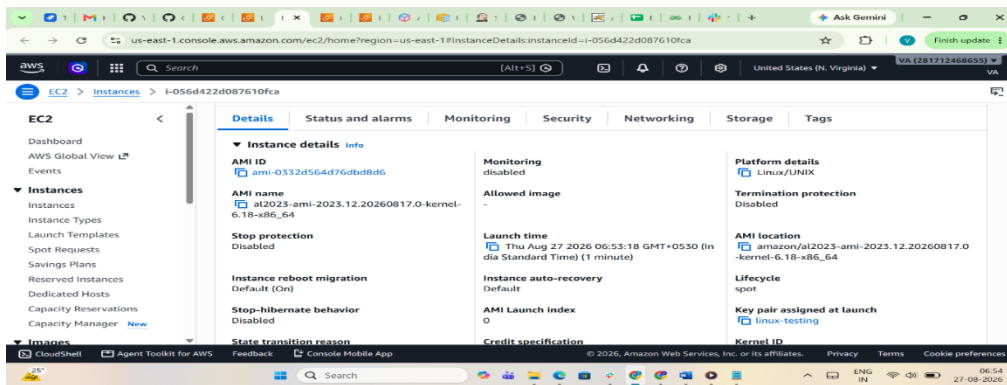
```

6. Launch any EC2 using the spot purchasing option.

Implementation:

1. Selected an appropriate AMI and instance type for the Spot Instance.
2. Configured the required VPC, subnet, security group, and key pair.
3. Selected the Spot Instance purchasing option.
4. Configured the required Spot request settings.
5. Launched the Spot Instance.
6. Verified the instance state and Spot Instance request status.





Choose filter set Find Instance by attribute or tag (case-sensitive)

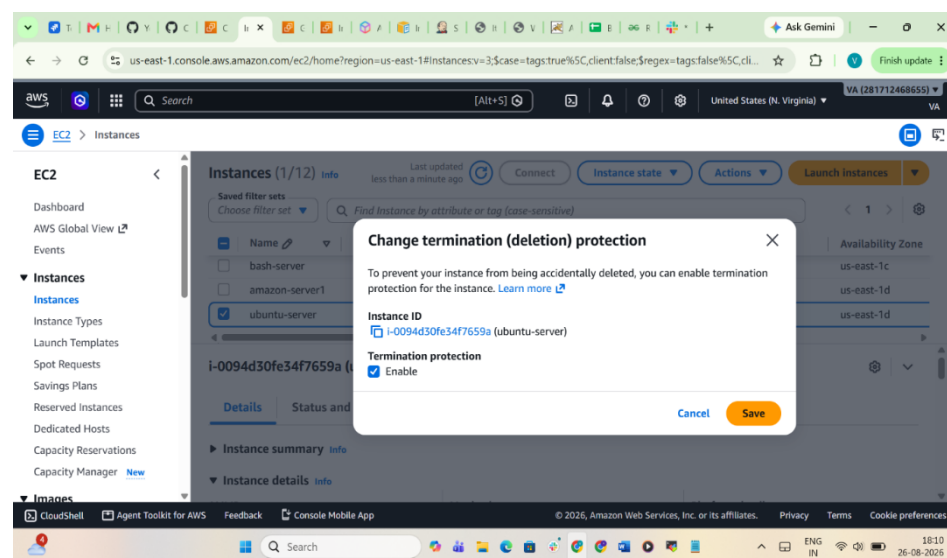
Instance state = running Clear filters

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
☐	spot-server	i-056d422d087610fca	Running	t3.micro	3/3 checks passed	us-east-1d

7. Enable termination policy on the EC2 created in Task 2.

Implementation:

1. Selected the Ubuntu EC2 instance created in Task 2.
2. Opened the instance configuration settings.
3. Enabled Termination Protection for the EC2 instance.
4. Verified that termination protection was enabled.
5. Attempted to terminate the instance to verify that termination was prevented.
6. Disabled termination protection when required to allow termination.




```
PS C:\WINDOWS\system32> aws ec2 describe-instances `
>> --instance-ids i-0c15f9182208daad `
>> --query "Reservations[0].Instances[0].(State.Name,PublicIpAddress,SubnetId,VpcId,SecurityGroups[*].GroupId)" `
>> --output table
>> --region ap-south-1
-----
DescribeInstances
-----
running
13.204.85.64
subnet-077298c93cb247c0e
vpc-01c2894d3e80f5ce9
sg-0092e2612298a64f2

PS C:\WINDOWS\system32> aws ec2 authorize-security-group-ingress `
>> --group-ids sg-0092e2612298a64f2 `
>> --protocol tcp `
>> --port 22 `
>> --cidr 49.204.26.15/32 `
>> --region ap-south-1
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-08e9ca98628308b91",
      "GroupId": "sg-0092e2612298a64f2",
      "GroupOwnerId": "281712468655",
      "IpAddress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "CidrIpv4": "49.204.26.15/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:ap-south-1:281712468655:security-group-rule/sgr-08e9ca98628308b91"
    }
  ]
}

PS C:\WINDOWS\system32> aws ec2 describe-security-groups `
>> --group-ids sg-0092e2612298a64f2
```

```
PrivateIpAddress": "172.31.8.158",
"PrivateIpAddresses": [
  {
    "Primary": true,
    "PrivateDnsName": "ip-172-31-8-158.ap-south-1.compute.internal",
  }
]
^C
PS C:\WINDOWS\system32> aws ec2 describe-instances `
>> --filters "Name=instance-state-name,Values=pending,running" `
>> --query "Reservations[*].Instances[*].(InstanceId,State.Name,InstanceType,PrivateIpAddress,PublicIpAddress)" `
>> --output table
>> --region ap-south-1
-----
DescribeInstances
-----
i-0c15f9182208daad | running | t3.micro | 172.31.8.158 | 13.204.85.64
```

```
PS C:\WINDOWS\system32> aws ec2 describe-images --owners amazon --filters "Name=name,Values=al2023-ami-2023*-x86_64" "Name=state,Values=available" --query "Images | sort_by($, &[start-date])" --output table --region ap-south-1
-----
DescribeImages
-----
ami-0f48d8eda3141f58a
al2023-ami-2023-11-20260817-0-kernel-6.12-x86_64

PS C:\WINDOWS\system32> aws ec2 run-instances `
>> --image-id ami-0f48d8eda3141f58a `
>> --instance-type t3.micro `
>> --key-name linux-testing `
>> --security-group-ids sg-0092e2612298a64f2 `
>> --count 1 `
>> --region ap-south-1

aws: [ERROR]: An error occurred (InvalidParameterCombination) when calling the RunInstances operation: The specified instance type is not eligible for Free Tier. For a list of Free Tier instance types, run 'describe-instance-types' with the filter 'free-tier-eligible=true'.
PS C:\WINDOWS\system32> aws ec2 run-instances `
>> --image-id ami-0f48d8eda3141f58a `
>> --instance-type t3.micro `
>> --key-name linux-testing `
>> --security-group-ids sg-0092e2612298a64f2 `
>> --count 1 `
>> --region ap-south-1
{
  "ReservationId": "r-0b664c12155780b93",
  "OwnerId": "281712468655",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "ca779bb4-71b5-45f9-a888-b58c8aeb5c14",
      "EbsOptimized": false,
      "EnaSupport": true,

```

```
PS C:\WINDOWS\system32> ssh-keygen -y -f "$env:USERPROFILE\Downloads\linux-testing.pem" > "$env:USERPROFILE\Downloads\linux-testing.pub"
PS C:\WINDOWS\system32> Get-Content "$env:USERPROFILE\Downloads\linux-testing.pub"
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCTxQ5InIUBVjQM0t2awU89K9PuyZCXvpg1GKZ+8LZapJrS9ZIDBhCt2mNwWYTSxJgNqCkOfz+Z8KJBY3McFnrya9JgMB0UHA1z4R9Brt0AwVgXjRw/1XF7ouc8kDud
VPBairVieAt8v6uB1DMCjXbX+TA/2025w1j8uogrUFwWSZK/j629d0U18tNqei2t+mZdx1tAEaPqHNZ+dMcFbwgILjgde/D4LW5yF7tm92ojennT4r6/9BUJVt9PkEvyZedGZannOZwcAshd/27kzp8R/A2cyCJkXzkB07T6/z
8gX9ZtFw87Nifrb5G6wJcoMQPRh1QZKT697
PS C:\WINDOWS\system32> aws ec2 import-key-pair --key-name linux-testing --public-key-material fileb://"$env:USERPROFILE\Downloads\linux-testing.pub" --region ap-south-1

aws: [ERROR]: An error occurred (InvalidParameterValue) when calling the ImportKeyPair operation: Value for parameter PublicKeyMaterial is invalid. Character sets beyond A
CII are not supported.
PS C:\WINDOWS\system32> $PublicKey = Get-Content "$env:USERPROFILE\Downloads\linux-testing.pub" -Raw
PS C:\WINDOWS\system32> $PublicKey = $PublicKey.Trim()
PS C:\WINDOWS\system32> $PublicKey
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCTxQ5InIUBVjQM0t2awU89K9PuyZCXvpg1GKZ+8LZapJrS9ZIDBhCt2mNwWYTSxJgNqCkOfz+Z8KJBY3McFnrya9JgMB0UHA1z4R9Brt0AwVgXjRw/1XF7ouc8kDud
VPBairVieAt8v6uB1DMCjXbX+TA/2025w1j8uogrUFwWSZK/j629d0U18tNqei2t+mZdx1tAEaPqHNZ+dMcFbwgILjgde/D4LW5yF7tm92ojennT4r6/9BUJVt9PkEvyZedGZannOZwcAshd/27kzp8R/A2cyCJkXzkB07T6/z
8gX9ZtFw87Nifrb5G6wJcoMQPRh1QZKT697
PS C:\WINDOWS\system32> aws ec2 import-key-pair --key-name linux-testing --public-key-material $PublicKey --region ap-south-1

aws: [ERROR]: Invalid base64: "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCTxQ5InIUBVjQM0t2awU89K9PuyZCXvpg1GKZ+8LZapJrS9ZIDBhCt2mNwWYTSxJgNqCkOfz+Z8KJBY3McFnrya9JgMB0U
1z4R9Brt0AwVgXjRw/1XF7ouc8kDudVPBairVieAt8v6uB1DMCjXbX+TA/2025w1j8uogrUFwWSZK/j629d0U18tNqei2t+mZdx1tAEaPqHNZ+dMcFbwgILjgde/D4LW5yF7tm92ojennT4r6/9BUJVt9PkEvyZedGZannOZw
PS C:\WINDOWS\system32> $PublicKeyBytes = [System.Text.Encoding]::ASCII.GetBytes($PublicKey)
PS C:\WINDOWS\system32> $PublicKeyBase64 = [Convert]::ToBase64String($PublicKeyBytes)
PS C:\WINDOWS\system32> aws ec2 import-key-pair --key-name linux-testing --public-key-material $PublicKeyBase64 --region ap-south-1
{
  "KeyFingerprint": "1f:44:68:2c:6f:fd:92:09:e5:12:35:1c:4e:47:5c:8e",
  "KeyName": "linux-testing",
  "KeyPairId": "key-068bbcfce3886ecda"
}
PS C:\WINDOWS\system32> aws ec2 describe-key-pairs --query "KeyPairs[*].KeyName" --output table --region ap-south-1
-----
DescribeKeyPairs
-----
linux-testing
```

```

AWS Access Key ID [*****]: AKIAUOF27EKXZVKDPAMT
AWS Secret Access Key [*****]: ToRcJXicUDSA+n3oMowd9+aOVZaTztetC45VDqS
PS C:\WINDOWS\system32> aws configure
AWS Access Key ID [*****]: AKIAUOF27EKXZVKDPAMT
AWS Secret Access Key [*****]: ToRcJXicUDSA+n3oMowd9+aOVZaTztetC45VDqS
Default region name [None]: ap-south-1
Default output format [None]: json
PS C:\WINDOWS\system32> aws sts get-caller-identity
{
  "UserId": "281712468655",
  "Account": "281712468655",
  "Arn": "arn:aws:iam::281712468655:root"
}

PS C:\WINDOWS\system32> aws ec2 describe-images --owners amazon --filters "Name=name,Values=al2023-ami-*" "Name=state,Values=available" --query "Images | sort_by(@, &CreateDate)[-1].[ImageId,Name]" --output table --region ap-south-1
-----
DescribeImages
-----
|
| ami-0c562b0a28fe197f3 |
| al2023-ami-ecs-neuron-hvm-2023.0.20260820-kernel-6.1-x86_64 |
|
-----

PS C:\WINDOWS\system32> aws ec2 describe-key-pairs --query "KeyPairs[*].KeyName" --output table --region ap-south-1
PS C:\WINDOWS\system32> aws ec2 describe-security-groups --query "SecurityGroups[*].[GroupId,GroupName]" --output table --region ap-south-1
-----
DescribeSecurityGroups
-----
|
| sg-0092e2612298a64f2 | default |
|
-----

```

```

PS C:\WINDOWS\system32> aws ec2 describe-instances `
>> --instance-ids i-0c15f91822008daad `
>> --query "Reservations[0].Instances[0].[InstanceId,State.Name]" `
>> --output table `
>> --region ap-south-1
-----
DescribeInstances
-----
+-----+
| i-0c15f91822008daad |
| stopped             |
+-----+

```